LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – Inheritance, Static, and Collection



Keyla Azzahra

103132400017

S1SD-05

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Inheritance (Pewarisan)

   Inheritance adalah mekanisme dalam pemrograman berorientasi objek yang memungkinkan suatu kelas (subclass/kelas turunan) mewarisi properti (atribut) dan perilaku (method) dari kelas lain (superclass/kelas induk). Konsep ini mendukung *reusability* kode. Kata kunci extends digunakan untuk menurunkan kelas, dan super digunakan untuk merujuk pada konstruktor atau method dari kelas induk.

2. Method Overriding & Overloading
   a) Method Overriding: Terjadi ketika subclass mendefinisikan ulang method yang sudah ada di superclass dengan nama, parameter, dan tipe kembalian yang sama.
   b) Method Overload: Terjadi ketika dalam satu kelas terdapat beberapa method dengan nama yang sama tetapi memiliki parameter yang berbeda (jumlah atau tipe data).

3. Static Modifier
   a) Static Variables: Variabel yang dimiliki oleh kelas itu sendiri, bukan oleh objek individual. Nilainya dibagi bersama oleh semua objek dari kelas tersebut.
   b) Static Methods: Method yang dapat dipanggil langsung melalui nama kelasnya tanpa harus membuat instance (objek) terlebih dahulu.

4. Java Collection & Generics

   Collection Framework menyediakan struktur data yang dinamis untuk menyimpan dan memanipulasi sekelompok objek (seperti List, Set, Queue). Penggunaan *Generics* (<T>) memastikan keamanan tipe data (*type safety*) pada koleksi tersebut tanpa perlu *casting* manual.

B. Guided

1. Inheritance Dasar

A.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package guided1;

/**
 *
 * @author user
```

```
 */
public class A {

    int x;
    int y;

    void tampilXY() {
        System.out.println("Nilai x: " + x + " nilai y: " + y);
    }
}
```

B.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided1;

/**
 *
 * @author user
 */
public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ() {
        hasil = x+y+z;

        System.out.println("jumlah (x + y + z) = " + hasil);
    }
}
```

Demo.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
```

```
 */
package guided1;

/**
 *
 * @author user
 */
public class Demo {
   public static void main(String[] args) {
      A superclass = new A();
      B subclass = new B();

      superclass.x = 3;
      superclass.y = 4;

      System.out.println("Superclass: ");
      superclass.tampilXY();


      // member superclass dapat diakses dari subclassnya
      subclass.x = 1;
      subclass.y = 2;
      subclass.z = 5;

      System.out.println("Subclass: ");
      subclass.jumlahXYZ();

   }
}
```

Output :

```
--- exec:3.0.3:exec (default-cli) @ Modul6 ---
Superclass:
Nilai x: 3 nilai y: 4
Subclass:
jumlah (x + y + z) = 8
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Inheritance dengan Constructor & Super

Employee.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package guided2;

/**
 *
 * @author user
 */
public class Employee {
    private String name;
    String departemen;

    public Employee(String nama) {
        this.name = nama;
    }

    public void tampilNama() {
        System.out.println("nama : " + name);
    }
}
```

Manager.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided2;

/**
 *
 * @author user
 */
public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String alamat) {
```

```
        /* memanggil konstruktor employee */
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi() {
        super.tampilNama();

        /* menampilkan variabel alamat dan departemen */
        System.out.println("alamat : " + alamat);
        System.out.println("departemen : " + super.departemen);
    }
}
```

DemoManager.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package guided2;

/**
 *
 * @author user
 */
public class DemoManager {
    public static void main(String[] args) {
        /* membuat objek */
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
nama : Adi
alamat : Sukabirus
departemen : SDM
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Java Collection / ArrayList

Lagu.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package guided3;

/**
 *
 * @author user
 */
public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void Tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package guided3;

/**
 *
 * @author user
```

```
 */
public class Main {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("Lagu 2", "aku");

        d.Tampilkan();
        e.Tampilkan();
    }
}
```

Output :

```
] --- exec:3.6.3:exec (default-cli) @ modul6 ---
  Judul : Lagu 1, pencipta : Tidak dikenal
  Judul : Lagu 2, pencipta : aku
  ------------------------------------------------------------------------
  BUILD SUCCESS
  ------------------------------------------------------------------------
```

4. Method Overriding & Polimorfisme

A.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided4;

/**
 *
 * @author user
 */
public class A {

    public void tampilkan() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

B.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided4;

/**
 *
 * @author user
 */
public class B extends A {

    public void tampilkan() {
        super.tampilkan(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

Main.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package guided4;

/**
 *
 * @author user
 */
public class Main {

    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ modul6 ---
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa. Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Jawaban :

ModelML.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package unguided;

/**
 *
 * @author user
 */
public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public String getNamaModel() {
        return namaModel;
    }
```

```
    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel + " | Data latih: " +
jumlahDataLatih);
    }

    // Method overloading: Hitung biaya tanpa parameter
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // Method overloading: Hitung biaya dengan satu parameter (epoch)
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

ModelRegresi.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package unguided;

/**
 *
 * @author user
 */
public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int
jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public int getJumlahFitur() {
        return jumlahFitur;
    }
```

```
    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Method baru dengan dua parameter (epoch dan ukuran batch)
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return (getJumlahDataLatih() * epoch) / ukuranBatch;
    }
}
```

ModelKlasifikasi.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package unguided;

/**
 *
 * @author user
 */
public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int
jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    public int getJumlahKelas() {
        return jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
```

```
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

ModelDeepLearning

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package unguided;

/**
 *
 * @author user
 */
public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

Main.java

```
        /*
         * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
        default.txt to change this license
         * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
        edit this template
         */
        package unguided;

        /**
         *
         * @author user
         */
        public class Main {
            public static void main(String[] args) {
                ModelML[] daftarModel = new ModelML[3];
                daftarModel[0] = new ModelRegresi("Regresi Linier", 500, 4);
                daftarModel[1] = new ModelKlasifikasi("Decision Tree", 800, 3);
                daftarModel[2] = new ModelDeepLearning("Neural Network", 1000,
        5, 3);

                int totalBiaya = 0;

                for (ModelML m : daftarModel) {
                    m.tampilkanInfo();
                    int biaya = m.hitungBiaya();
                    System.out.println("Biaya komputasi: " + biaya);
                    System.out.println("---");
                    totalBiaya += biaya;
                }

                System.out.println("Total biaya: " + totalBiaya);
                System.out.println("Biaya 10 epoch (Neural Network): " +
        daftarModel[2].hitungBiaya(10));
                System.out.println("Biaya 10 epoch (Regresi Linier): " +
        daftarModel[0].hitungBiaya(10));
                System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
        ((ModelRegresi) daftarModel[0]).hitungBiaya(10, 50));
            }
}
```

Output :

```
] --- exec:3.6.3:exec (default-cli) @ modul6 ---
 Model: Regresi Linier | Data latih: 500
 Jumlah fitur: 4
 Biaya komputasi: 1040
 ---
 Model: Decision Tree | Data latih: 800
 Jumlah kelas: 3
 Biaya komputasi: 4800
 ---
 Model: Neural Network | Data latih: 1000
 Jumlah kelas: 5
 Jumlah layer: 3
 Biaya komputasi: 10600
 ---
 Total biaya: 16440
 Biaya 10 epoch (Neural Network): 10000
 Biaya 10 epoch (Regresi Linier): 5000
 Biaya 10 epoch batch 50 (Regresi Linier): 100
 ------------------------------------------------------------------------
 BUILD SUCCESS
 ------------------------------------------------------------------------
```

Penjelasan :

Program di atas menerapkan konsep *inheritance* berjenjang (multilevel inheritance) dan hierarkis. Kelas ModelML berperan sebagai *superclass* yang memiliki atribut privat namaModel dan jumlahDataLatih serta method hitungBiaya() yang di-*overload*.

a) ModelRegresi mewarisi ModelML dan menambahkan atribut jumlahFitur.
b) ModelKlasifikasi mewarisi ModelML dengan tambahan atribut jumlahKelas.
c) ModelDeepLearning mewarisi ModelKlasifikasi dengan tambahan atribut jumlahLayer.

Semua kelas melakukan *method overriding* pada tampilkanInfo() dan hitungBiaya() untuk menyesuaikan rumus perhitungan biayanya masing-masing, serta menggunakan polimorfisme melalui array bertipe ModelML.

D. Kesimpulan

Praktikum ini membuktikan bahwa konsep *Inheritance* (pewarisan), *Method Overriding*, dan *Method Overloading* memudahkan pengelolaan kode program Java menjadi lebih terstruktur dan modular. Penggunaan *encapsulation* (atribut privat), *polymorphism* (array kelas induk), serta keyword super sangat efektif dalam menyelesaikan kasus komputasi *machine learning* yang kompleks.